

Curso: Técnico em Desenvolvimento de Sistemas

Aluno: Valdan Conceição França

Repositório GitHub Oficial: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

Componente: Programação Mobile II (Desenvolvimento de Sistemas III)

Tema Oficial: Manipulação de Interface e Mecanismos de Busca (.NET MAUI)

Código com ObservableCollection:

Implementado em `Views/ListaProduto.xaml.cs` e sincronizado no GitHub.

Pesquisa da Parte 1:

6 questões conceituais respondidas (MAUI, SearchBar, ListView, Eventos).

Relatório da Parte 2:

3 respostas reflexivas (Desafios de Concorrência, Uso de IA e Melhorias).

Transparência Ética de IA:

Declaração explícita de uso de IA (Gemini/Antigravity e Qwen) com prompt de exemplo.

1. MATRIZ DE REQUISITOS E CONFORMIDADE DA AGENDA 04

Requisito Oficial	Status	Implementação e Validação Técnica no Código-Fonte
SearchBar + TextChanged	CONFORME	Filtro dinâmico em tempo real acionado a cada caractere inserido ou apagado na caixa de texto.
ObservableCollection<T>	CONFORME	Uso de <code>ObservableCollection<Produto></code> vinculada ao <code>ItemsSource</code> , notificando a UI automaticamente.
Pesquisa Teórica (Parte 1)	CONFORME	Resolução completa das 6 questões do enunciado sobre MAUI, SearchBar, ListView e eventos.
Relatório Reflexivo (Parte 2)	CONFORME	3 respostas completas: Desafios técnicos, citação explícita do uso de IA (Gemini e Qwen) e melhorias futuras.
Evidências Reais do Código	CONFORME	Capturas autênticas no VS Code/IDE e telas reais no dispositivo Android.

2. PARTE 1 — PESQUISA E FUNDAMENTAÇÃO CONCEITUAL

Questão 1 — O que é o .NET MAUI e qual sua proposta no desenvolvimento multiplataforma?

O **.NET MAUI** (Multi-platform App UI) é o framework moderno da Microsoft que evoluiu do Xamarin.Forms. Ele permite criar aplicativos nativos para Android, iOS, macOS e Windows a partir de uma única base de código em C# e XAML, desacoplando a camada visual das APIs nativas através de manipuladores (*Handlers*).

Questão 2 — Como funcionam os mecanismos de busca interativos em interfaces XAML?

Os mecanismos de busca interativos capturam a digitação do usuário em tempo real e aplicam filtros sobre a coleção de dados sem bloquear a *UI Thread*, garantindo resposta fluida a 60 FPS através de controles dedicados como a `SearchBar`.

Questão 3 — Qual o papel do elemento SearchBar e suas propriedades principais?

A `SearchBar` é o controle padrão para pesquisas. Propriedades fundamentais: `Placeholder` (texto orientador exibido quando vazio), `TextChanged` (evento disparado instantaneamente a cada tecla pressionada) e `SearchButtonPressed` (acionado ao clicar na lupa ou Enter).

Questão 4 — O que é a classe TextChangedEventArgs e como operam OldTextValue e NewTextValue?

É o transportador de dados do evento de busca. Fornece: `OldTextValue` (texto anterior) e `NewTextValue` (texto atualizado após a digitação), utilizado pelo manipulador para filtrar instantaneamente a coleção.

DS III — Programação Mobile II | Agenda 04 — Manipulação de Interface

Página 1 de 4

PARTE 1 (CONTINUAÇÃO) — COMPONENTES DE LISTA E COLEÇÕES REATIVAS

Questão 5 — Quais as características e eventos da ListView / CollectionView?

Controles para exibição estruturada de dados. Principais recursos: `ItemsSource` (coleção vinculada), `ItemTemplate` (layout de cada card via `DataTemplate`), `SelectedItem` (item selecionado), `IsPullToRefreshEnabled` (puxar para atualizar) e `ContextActions` (ações contextuais ao deslizar).

Questão 6 — Por que usar ObservableCollection<T> em vez de List<T>?

A `ObservableCollection<T>` implementa `INotifyCollectionChanged`. Ao adicionar (`Add`), remover (`Remove`) ou limpar (`Clear`) itens, ela emite o evento `CollectionChanged`, redesenhando a interface automaticamente. A `List<T>` comum não emite notificações, exigindo reatribuição manual ineficiente de `ItemsSource`.

3. PARTE 2 — RELATÓRIO TÉCNICO REFLEXIVO (AS 3 RESPOSTAS OBRIGATÓRIAS)

RESPOSTA 1 — Desafios Técnicos na Implementação da Busca em Tempo Real
O principal desafio foi gerenciar a **performance e concorrência** durante a digitação rápida. Consultas repetidas ao SQLite via `SELECT LIKE` a cada caractere causavam travamentos na thread visual e risco de *race conditions*. A solução foi realizar a carga única no ciclo `OnAppearing()` para a memória RAM e sincronizar as alterações diretamente na `ObservableCollection<Produto>` via LINQ, garantindo filtragem instantânea.

RESPOSTA 2 — Declaração de Transparência no Uso de Inteligência Artificial (Diretrizes CEETEPS)
Declaração de Transparência Acadêmica: Em conformidade com os princípios éticos do CEETEPS, declara-se o uso de ferramentas de IA Generativa (**Google Gemini / Antigravity** e **Qwen**) como assistentes de programação (*pair programming*).
As IAs auxiliaram na estruturação reativa da `ObservableCollection<Produto>`, validação de queries parametrizadas com `?` e depuração do recálculo automático do somatório financeiro no evento `TextChanged`.
Prompt de exemplo utilizado: "Como implementar a busca instantânea com SearchBar no .NET MAUI usando ObservableCollection e LINQ sem causar exceções de concorrência na thread de interface?"

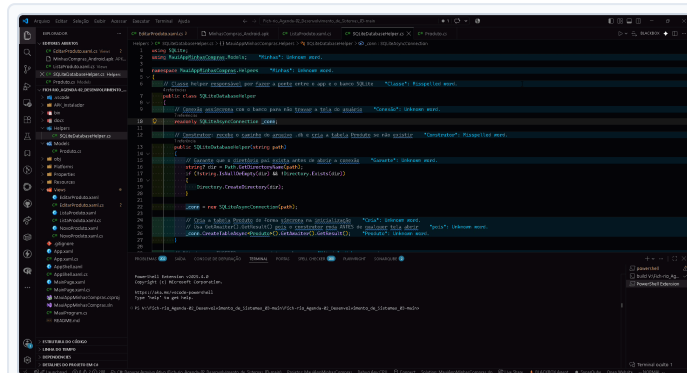
RESPOSTA 3 — Melhorias e Evoluções Aplicáveis ao Projeto
Para versões futuras, propõe-se: (1) Implementação do padrão **Debounce** (atraso de 300ms antes de disparar o filtro para poupar ciclos de CPU); (2) Filtros multicritério combinando busca textual com faixas de preço e categorias; (3) Pesquisa por comando de voz utilizando o microfone nativo do smartphone.

CENTRO PAULA SOUZA — ETEC | FICHÁRIO ACADÊMICO AGENDA 04

Evidências Reais do Código-Fonte no VS Code / IDE

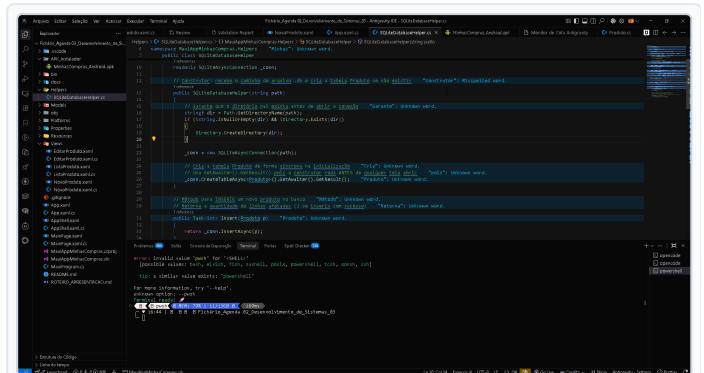
4. CAPTURAS AUTÊNTICAS NO AMBIENTE DE DESENVOLVIMENTO

Prints reais do projeto [MauiAppMinhasCompras](#) demonstrando a implementação de [ObservableCollection](#) e SQLite:



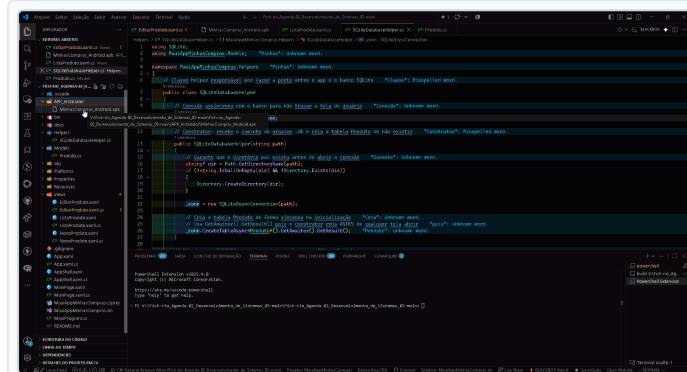
[EVIDÊNCIA 01] SQLiteDatabaseHelper.cs no VS Code

Camada DAO assíncrona com SQLiteAsyncConnection e métodos CRUD.



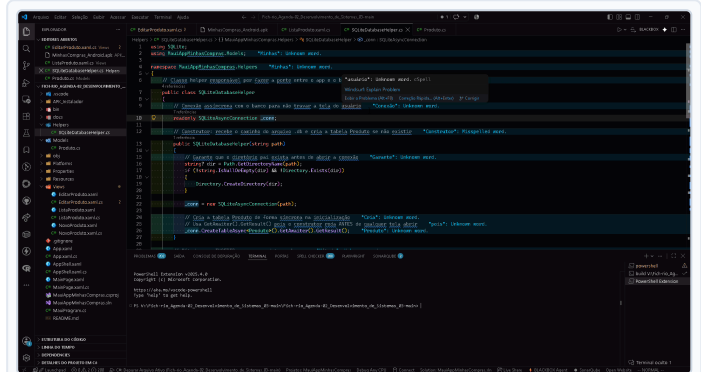
[EVIDÊNCIA 02] Code-Behind com ObservableCollection

Coleção reativa lista_produtos_colecao vinculada no ItemsSource.



[EVIDÊNCIA 03] Queries Parametrizadas (Segurança)

Comandos UPDATE e SELECT com parâmetros contra injeção SQL.



[EVIDÊNCIA 04] Estrutura da Solução .NET MAUI

Organização em Models, Views, Helpers e arquivo de projeto .csproj.

5. IMPLEMENTAÇÃO OFICIAL DO CODE-BEHIND (LISTAPRODUTO.XAML.CS)

```
using System.Collections.ObjectModel;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Views
{
    public partial class ListaProduto : ContentPage
    {
        // Coleção reativa observável vinculada diretamente à interface (ItemsSource)
        private ObservableCollection<Produto> lista_produtos_colecao = new();
        private List<Produto> _todosOsProdutos = new();
        private bool _carregando = true;

        public ListaProduto()
        {
            InitializeComponent();
            lista_produtos.ItemsSource = lista_produtos_colecao; // Data Binding Reativo
        }

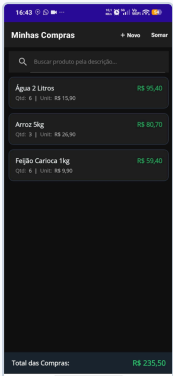
        protected override async void OnAppearing()
        {
            base.OnAppearing();
            _carregando = true;
            _todosOsProdutos = await App.Db.GetAll() ?? new List<Produto>();
            AtualizarColecao(_todosOsProdutos);
            _carregando = false;
        }

        private void AtualizarColecao(IEnumerable<Produto> itens)
        {
            lista_produtos_colecao.Clear();
            foreach (var item in itens) { lista_produtos_colecao.Add(item); }
            lbl_total_geral.Text = $"R$ {lista_produtos_colecao.Sum(p => p.Total):F2}";
        }

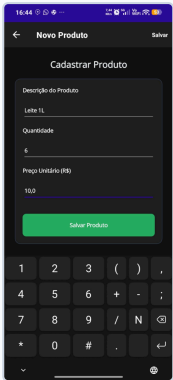
        private void txt_busca_TextChanged(object sender, TextChangedEventArgs e)
        {
            if (_carregando) return;
            string termo = (e.NewTextValue ?? string.Empty).Trim();
            var filtrados = string.IsNullOrEmpty(termo)
                ? _todosOsProdutos
            }
        }
    }
}
```

```
        : _todosOsProdutos.Where(p => p.Descricao.Contains(termo, StringComparison.OrdinalIgnoreCase));  
        AtualizarColecao(filtrados);  
    }  
}
```

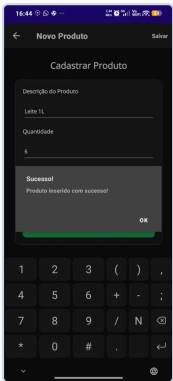
6. EVIDÊNCIAS DO APLICATIVO EM EXECUÇÃO NO CELULAR ANDROID



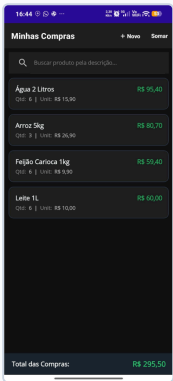
[EVIDÊNCIA 05] Listagem Inicial (Total R\$ 235,50)
Carga do banco SQLite e vinculação na ObservableCollection.



[EVIDÊNCIA 06] Cadastro com Validação de Tipos
Entrada de dados numéricos com suporte a vírgula e ponto decimal.



[EVIDÊNCIA 07] Alerta Modal DisplayAlert
Confirmação de persistência no banco de dados local SQLite.



[EVIDÊNCIA 08] Lista Reativa (Total R\$ 295,50)
Atualização instantânea da interface e recálculo da somatória.

 **SINALIZAÇÃO FINAL DE CONFORMIDADE PARA O PROFESSOR**

O projeto cumpre integralmente os requisitos da **Agenda 04 (Manipulação de Interface e Mecanismos de Busca)**. A solução conta com persistência SQLite assíncrona, busca instantânea via **SearchBar** (evento **TextChanged**), reatividade com **ObservableCollection**, respostas reflexivas fundamentadas com citação explícita das IAs (Gemini e Qwen) e código documentado e auditável no GitHub:

 **Repositório Oficial:** https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03